PONTIFICIA UNIVERSIDAD JAVERIANA – CALI

FACULTAD DE INGENIERÍA Y CIENCIAS

ASIGNATURA: ÁRBOLES Y GRAFOS – B

PROFESORES: CARLOS RAMÍREZ Y CARLOS DELGADO

Informe parcial del proyecto del curso

Realizado por Nicolás Carrillo R. y Nicolle Ureña G.

Repositorio asociado al proyecto: https://github.com/NicolleUrenaGarcia/AGRA_Project

I. Descripción y análisis general del enunciado

Se describe una realidad imaginaria de Mumm-Ra, un conquistador que, a lo largo de los años ha intentado conquistar la tercera tierra. Para lograrlo, debe capturar al líder de la oposición, Lion-O, a quien ya tiene localizado y está a punto de atacar.

Lion-O debe escapar de la tierra y para ello, debe dirigirse de alguna manera a las estaciones espaciales disponibles antes de que los mutantes aliados de Mumm-Ra lo atrapen.

Inicialmente, en el enunciado se puede observar que hay indicios claros de un grafo ponderado no dirigido al mencionar: “Una red de carreteras que conectan sus intersecciones”, “Cada carretera tiene una longitud conocida y los vehículos pueden circular por ellas en cualquiera de las dos direcciones”, por lo tanto, una representación de un grafo podría apuntar a que los vértices son las intersecciones y sus aristas no dirigidas son las carreteras.

Por otro lado, el problema busca evaluar la menor velocidad máxima sobre la que Lion-O puede garantizar su escape priorizando la optimización de la velocidad sobre el tiempo de llegada, en base a la estrategia de movimiento permitida a los mutantes

Ya que adicionalmente se menciona que se conocen las posiciones iniciales de Lion-O y Mumm-Ra, es posible modelar un grafo inicial para representar ambas posiciones.

Especificación del problema

Entradas:

Un grafo no dirigido G=(V, E) tal que V es el conjunto de intersecciones y E el conjunto de carreteras que conectan con las intersecciones con una función de peso $w : E \rightarrow \mathbb{R}_{>0}$

donde w(e) representa la longitud de una carretera en centenas de metros donde $e \in E$; Un entero v≥ 0, tal que v es la velocidad máxima en km/h de las naves de los mutantes; Un entero k ∈ $\mathbb{N}_0$ tal que k es la cantidad de salidas espaciales en el grafo; se cumple que $0 < k \leq |V|$; Dos vértices s y r tal que $s, r \in V$ que corresponden a las ubicaciones de Lion-O y los mutantes respectivamente; Un arreglo L[0…k) de nodos tal que $\forall i \in [0 \ldots k), L[i] \in V \wedge |L| = k$ que corresponden a los nodos donde están ubicadas las estaciones espaciales.

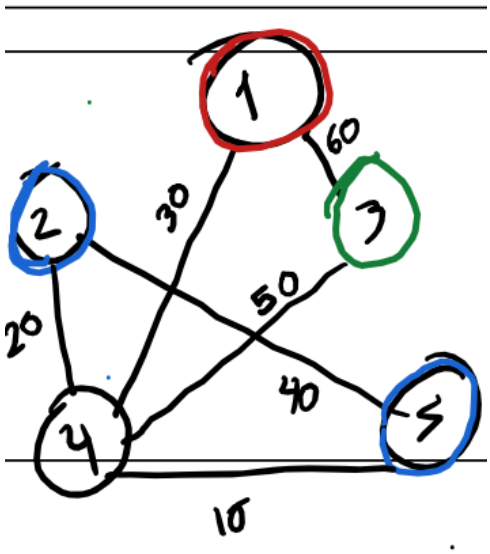
Salida: Un valor C, tal que $C \in \mathbb{R}^+$ donde C es la velocidad mínima necesaria que debe alcanzar Lion-O para escapar de los mutantes, ubicados en el vértice r, partiendo de su posición inicial s, desplazándose a través de las carreteras del grafo no dirigido $G = (V, E)$, cuyas longitudes están definidas por un costo e, y llegando a alguna de las k salidas espaciales ubicadas en los vértices almacenados en el arreglo L[0..k), tal que |L|=k, antes de ser alcanzado por los mutantes, cuya velocidad máxima es v.

Si C = -1, el programa debe imprimir “Mumm-Ra captures Lion-O!”, en caso contrario debe imprimir “Lion-O can escape at a speed of X.XXXXXX km/h!” donde X.XXXXXX representa el valor de C.

II. Casos de prueba (Input y output)

Primer caso de prueba. 5 vértices y 6 aristas. 2 salidas espaciales “Velocidades por segmento independiente”

Gráfica asociada:



INPUT:

1

5 6 2 80

1 3 60

1 4 30

4 3 50

2 5 40

2 4 20

4 5 10

3 1

2 5

OUTPUT:

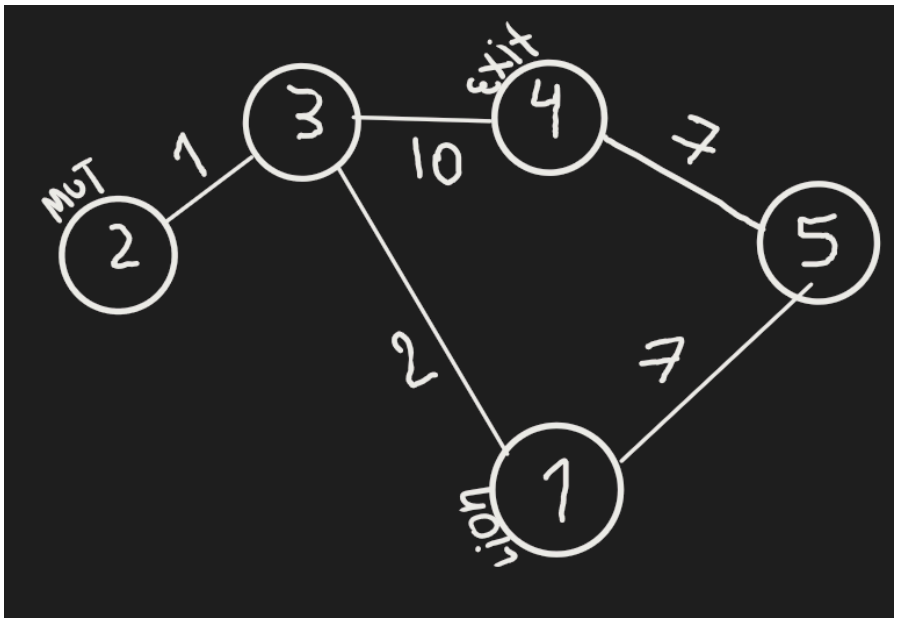
Lion-O can escape at a speed of 133.333333 km/h!

Lion inicia en el nodo 3 y los mutantes en el nodo 1, con salidas espaciales en 2 y 5. El resultado se obtiene en base al cálculo del nodo en el que Lion tiene mayor riesgo de ser capturado. Para dicho nodo (nodo 4), Lion debe tener una velocidad aún mayor que los Mutantes, pues tiene una distancia mayor que recorrer y se debe evaluar de manera óptima para que Lion logre llegar apenas antes que los mutantes para no ser capturado y

escapar. De igual manera, una vez supere este nodo que es una zona de riesgo, puede llegar a la salida si mantiene una velocidad incluso menor, no inferior ni igual a la de los mutantes, usar una velocidad superior a la obtenida sería innecesario, porque esta le garantiza el suficiente tiempo para adelantar a los mutantes.

Segundo caso de prueba. 5 Vertices, 5 Aristas, 1 salida espacial "El camino óptimo, no corto"

Gráfica asociada:



Input

1

5 5 1 50

1 3

1 5

5 4

4 3

3 2

1 2

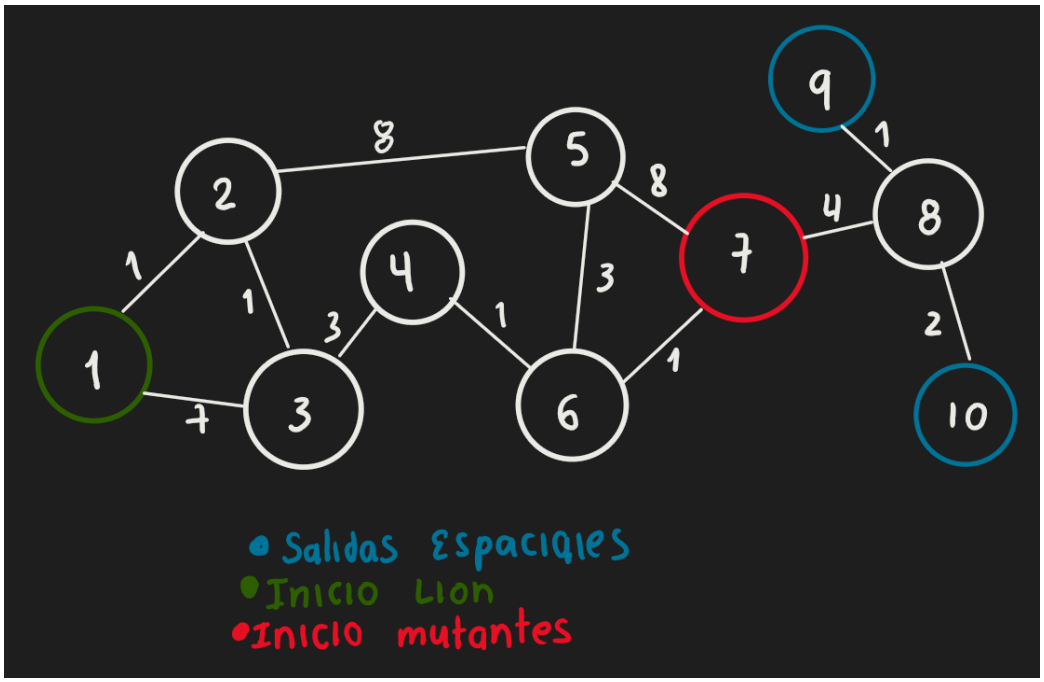
4

Output:

Lion-O can escape at a speed of 56.000000 km/h!

En este caso se evidencia un claro ejemplo que descartó una lógica anterior que planeábamos: El camino más corto es el más óptimo. Para este caso, existen 2 caminos por los cuales puede escapar Lion, sin embargo, el más corto (1-3-4) requiere una mayor exigencia con respecto a su velocidad máxima ya que el nodo de inicio de los mutantes tiene conexión directa e incluso más corta en distancia que Lion hacia la salida por dicho camino, por tanto se requiere una velocidad aún más exigente. Por el otro camino, se evidencia que a pesar que son distancias considerablemente mayores a las anteriores, los mutantes requerirían mayor tiempo para llegar hasta ahí, en este caso, requerirían atravesar en su camino más estratégico y rápido, los nodos 2, 3 y 1 para entrar en el camino de Lion atraviesa. Mientras que Lion ya habría adelantado en gran parte a los mutantes. Es por ello, que la velocidad requerida por este segundo camino es menor y la más óptima entre ambos caminos lo cual le garantiza una salida a Lion.

Tercer caso de prueba. 10 vértices y 12 aristas. 2 salidas espaciales "Cuello de botella"



INTPUT:

1

10 12 2 120

1 2 1

1 3 7

2 3 1

3 4 3

2 5 8

4 6 1

5 6 3

5 7 8

6 7 1

7 8 4

8 9 1

8 10 2

1 7

9 10

OUTPUT:

Mumm-Ra captures Lion-O!

Se plantea un caso que se específica más adelante en el documento: Nuestra hipótesis sobre los casos de captura. Es claro que cuando existan 2 o más caminos de intersecciones distintas con conexión directa hacia una salida espacial, con "distintos" nos referimos a que son independientes y no conducen a un cuello de botella en donde convergen. Con algunos casos de prueba realizados a papel y algunos evidenciados durante la bitácora, los casos en los que se captura a Lion es cuando el camino hacia la salida es único o convergen varios caminos y además es el nodo de inicio de los mutantes. Para este caso específico, es inutil buscar velocidades óptimas, pues los mutantes podrían permanecer en su nodo de inicio sabiendo que en algún momento llegaría Lion y capturarlo, pues estratégicamente saben que es su único acceso a la salida.

III. Consideraciones y retos

- La entrada de datos que determina la distancia entre intersecciones se da en enteros que deben leerse como centenas de metros. Es decir, si la distancia entre intersecciones es 7, entonces realmente son 700 metros en la distancia. Para ellos podríamos realizar la multiplicación por 100 una vez sean leídos por el programa.
- La entrada de distancias es en centenas de metros, mientras que la salida debe ser en km/h. Antes de entregar el resultado se deben realizar las conversiones respectivas
- Se puede implementar una estructura lineal, como una lista, con tantos nodos como tenga el grafo. En la posición de cada nodo habrá un identificador entero para marcar su función en el programa. 0 = Nodo normal, 1= Punto de partida de Lion-O, 2= Punto de partida de los mutantes, 3 = Estación espacial. Inicialmente todos son 0, a

medida que se lean las entradas se actualiza su tipo. (No es necesaria por ahora, pero no se descarta)

- Se necesita evaluar una forma de, en caso de existir, evaluar cual es el camino más óptimo entre varios caminos. Para ello se plantea la comparación del tiempo de llegada de los mutantes y Lion, sin embargo aún es un reto el factor por el cúal se evalúan las distintas variaciones de velocidad para reducir el tiempo. ¿Cómo se encuentra la velocidad óptima? ¿Es realmente la regla de 3 planteada una buena alternativa?
- Además, de no ser óptimo un camino aún es un reto el elemento de decisión para evaluar el siguiente camino a evaluar. ¿Se podría implementar un arreglo de arreglos con los caminos ideales? ¿Buscar caminos más cortos descartando un nodo donde llegan los mutantes antes y se requiere mayor velocidad? Es un componente fundamental para definir quien gana que todavía debemos perfeccionar
- Durante la introspección especificada en la bitácora, se evalúa el uso de un Dijkstra aplicado sobre los mutantes hacia todos los nodos del grafo y también deberíamos evaluar los tiempos de llegada a cada uno para compararlo con la llegada de Lion. Sin embargo, ¿de qué dependería la selección del nodo a comparar? ¿Quizá de todos los nodos del camino que está evaluando Lion hacia el final?
- Hemos evidenciado en algunos casos de prueba que generalmente las únicas formas de que Lion sea capturado es que uno de los nodos de el/los camino(s) que conducen a la(s) salida(s) espacial(es) sea el nodo de inicio de los mutantes y nos planteamos la hipótesis de que, siempre y cuando hayan varios nodos conectados a la salida espacial, no estén en un “cuello de botella” con los mutantes y Lion tenga acceso, siempre habrá una forma de que Lion escape, pues puede acelerar sin un límite determinado. Sin embargo, el hecho de que no hayamos encontrado un caso de prueba que contradiga nuestra hipótesis no significa que sea totalmente afirmativa, debemos evaluarla mejor durante el desarrollo del código.

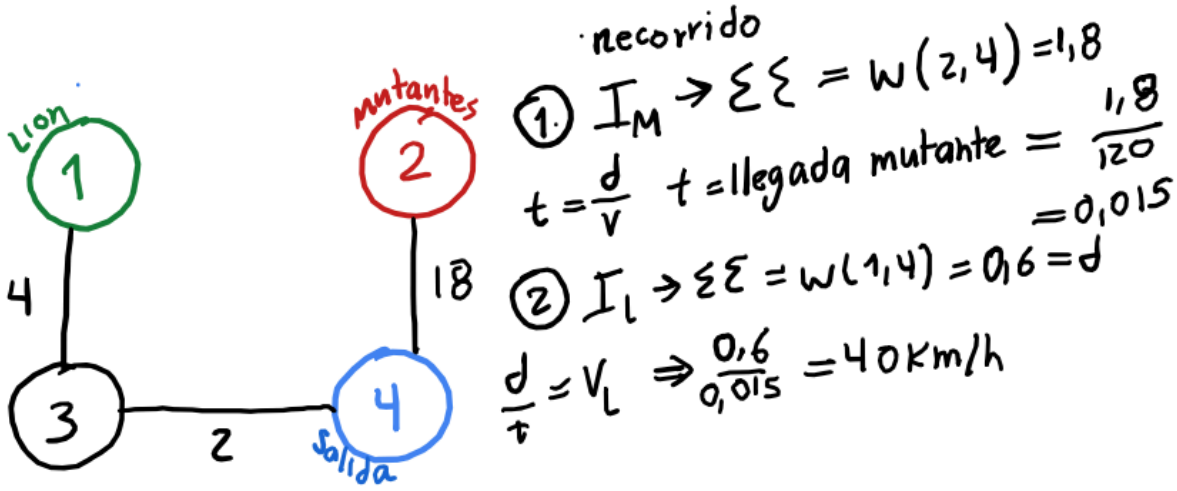
IV. Registros y avances (Bitácora)

30 de octubre - 2 de noviembre

Se realizan las primeras consignaciones del documento mediante la descripción y primer especificación del problema, así como el desarrollo de la primera intuición del enunciado. Se procesa el objetivo del proyecto, los casos de prueba y se plantea la primer especificación del problema.

3 de noviembre

En un primer análisis con los casos de prueba, se identificó que se podrían aplicar fórmulas físicas de distancias y velocidades, ya que el problema abarca datos relacionados con estos. Para encontrar la velocidad óptima de Lion-O, inicialmente y procesando los casos de prueba, tomamos la lógica intuitiva de que el objetivo de Lion es buscar un nodo que contenga la salida espacial más cercana para escapar en la menor velocidad posible de los mutantes. Para ello, asumimos en principio que se hace un algoritmo de caminos más cortos por parte de Lion hacia la salida más cercana y por parte de los mutantes hacia la salida a la que irá Lion (sin especificar qué algoritmo todavía). Bajo esta idea de que ambos necesitan llegar al mismo nodo por el camino más corto, se toma luego el hecho de cuál es la mínima velocidad para que Lion llegue a este vértice antes de que los mutantes lleguen. Por tanto se aplica la fórmula física de la distancia d=v*t y se sigue como lo aplicado en el TC3 observado en la imagen:



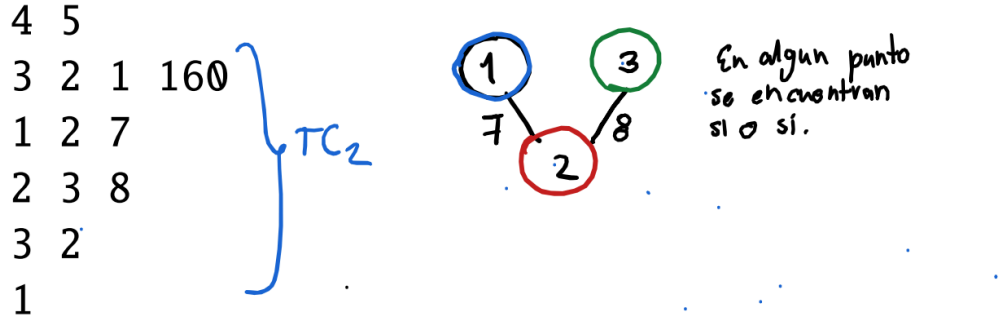
(Nota: Ignorar la línea 600 * X = 40, más que indicar la fórmula física en sí, indica los datos conocidos. El razonamiento por el cual se procedió está en los siguientes pasos)

Paso 1: Ya que los mutantes necesitan llegar estratégicamente al mismo punto que Lion-O, calculamos el camino más corto entre los nodos desde el inicio hasta la salida espacial, es 1800.

Según la fórmula de física d = v*t despejando t: t = d/v, despejamos el tiempo de llegada de los mutantes, el resultado es 15. Esto nos servirá para despejar velocidad en la fórmula de Lion. (En este punto se asumió que sí Lion y los mutantes llegaban al mismo tiempo a la estación espacial, Lion igualmente conseguía escapar)

Paso 2: Obtenemos igualmente la distancia mínima de Lion hacia la estacion espacial más cercana, en este caso es 600.

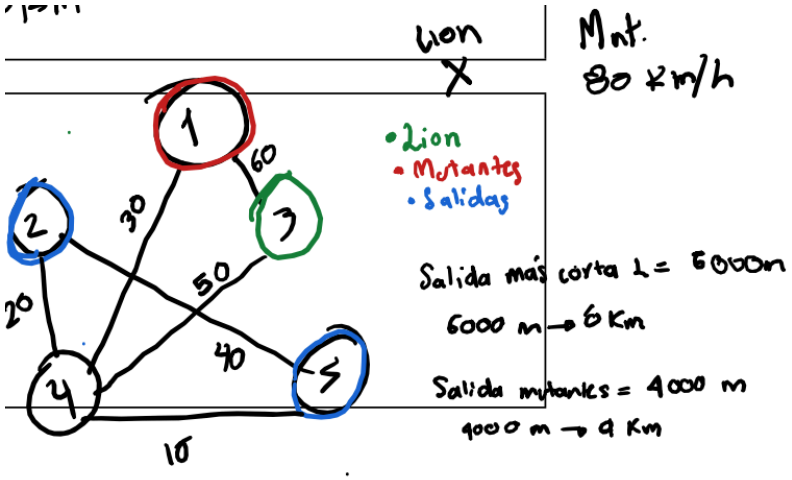
Procedemos a despejar v con d = 600 y t = 15. El resultado debería coincidir con el output propuesto



4 de noviembre:

Analizando de nuevo el problema y replanteando lo razonado de forma ordenada en una tabla, específicamente con los datos del tercer caso de prueba, notamos que, con una de regla de tres podríamos resolver el problema para la realización del ejercicio, con sus respectivas conversión de unidades y que, este razonamiento coincidía exitosamente con el output.

Adicionalmente, siguiendo la lógica anterior de la regla de tres, proporcionamos el siguiente caso de prueba:



5 de noviembre:

-En la primera implementación, se realiza la construcción de la lista de adyacencia siguiendo los parámetros del input propuesto. Se requiere un grafo no dirigido, puesto a la bidireccionalidad de las carreteras descritas en el enunciado; por tanto se agregan conexiones entre vértices en ambas

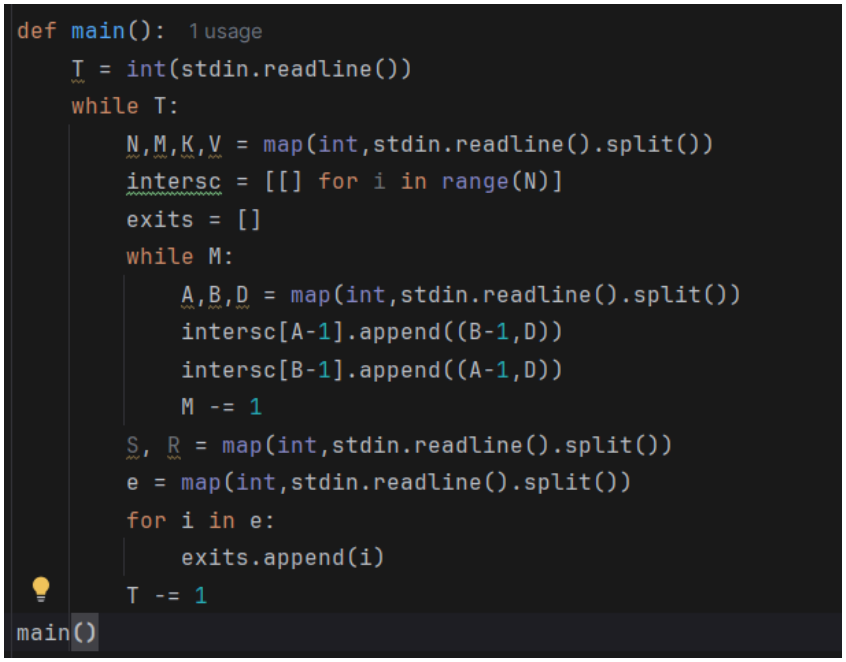
direcciones. De igual manera se dispone la lectura del input siguiendo la estructura propuesta: la primera línea equivale a la cantidad de casos de prueba, las siguientes con el número de vértices, aristas, número de salidas y la velocidad de los mutantes.

En las siguientes líneas con las conexiones entre nodos, las ubicaciones de inicio de Lion-O y los mutantes y finalmente, las ubicaciones de las salidas espaciales….

Posteriormente, realizamos una retroalimentación con el profesor, en donde nos hizo las siguientes observaciones:

- Se plantea que usar la misma velocidad para todo el recorrido a la estación espacial no garantiza que pueda escapar realmente de los mutantes
- A pesar de que la notación en la especificación del problema está bien orientada, se debe mejorar. Para la salida, los elementos deben expresarse en función de los de la entrada
- Las velocidades en los tramos pueden cambiar pues tanto los mutantes como Lion pueden acelerar y desacelerar según les convenga, sin embargo, el resultado será la velocidad máxima que usó Lion durante su recorrido sin haber sido capturado
- No existe forma alguna de que un peso sea negativo
- Los mutantes actúan como un solo actor, por tanto no deberían dividirse o distribuirse por distintos nodos del grafo
- La velocidad de los mutantes del input es tomada en km/h
- Durante la realización del proyecto se requieren realizar algunas conversiones puesto a que todas las unidades dadas no están en el mismo sistema de medición
- Las estaciones espaciales pueden ser la misma casilla de inicio de Lion o de los mutantes, existiendo la posibilidad de un grafo de 2 nodos
- Es necesario hacer más casos de prueba que rompan la lógica equivoca de “El camino más corto es el más óptimo, y por tanto, la velocidad para llegar a este es la que debe entregar el input”

Al haber redefinido la orientación de nuestras primeras hipótesis e intuiciones sobre la solución con ayuda del profesor, decidimos empezar implementando la lógica de aquellas que fueron confirmadas o se alinean a las especificaciones y requerimientos brindados. Por ello, empezamos implementando la lectura de los datos y la construcción de la lista de adyacencia necesaria para modelar nuestro grafo.



```
def main():  1 usage
    T = int(stdin.readline())
    while T:
        N,M,K,V = map(int,stdin.readline().split())
        intersc = [[] for i in range(N)]
        exits = []
        while M:
            A,B,D = map(int,stdin.readline().split())
            intersc[A-1].append((B-1,D))
            intersc[B-1].append((A-1,D))
            M -= 1
        S, R = map(int,stdin.readline().split())
        e = map(int,stdin.readline().split())
        for i in e:
            exits.append(i)
        T -= 1
main()
```

Además, nos planteamos nuevos retos en base a las hipótesis confirmadas por el profesor, como lo son:

- Ya que los mutantes pueden iniciar en la misma casilla que Lion, esto garantiza una pérdida inmediata. Se requiere elaborar un condicional donde se evalúen los siguientes casos:

- Si la posición de inicio de Lion-O es igual a una salida (ganando y arrojando velocidad de 0 Km/h).

-Si la posición de inicio de los mutantes es igual a una salida y esta es la única, de igual manera perderá, puesto a que los mutantes no requieren moverse, Lion no tiene opción y llegará en algún momento.

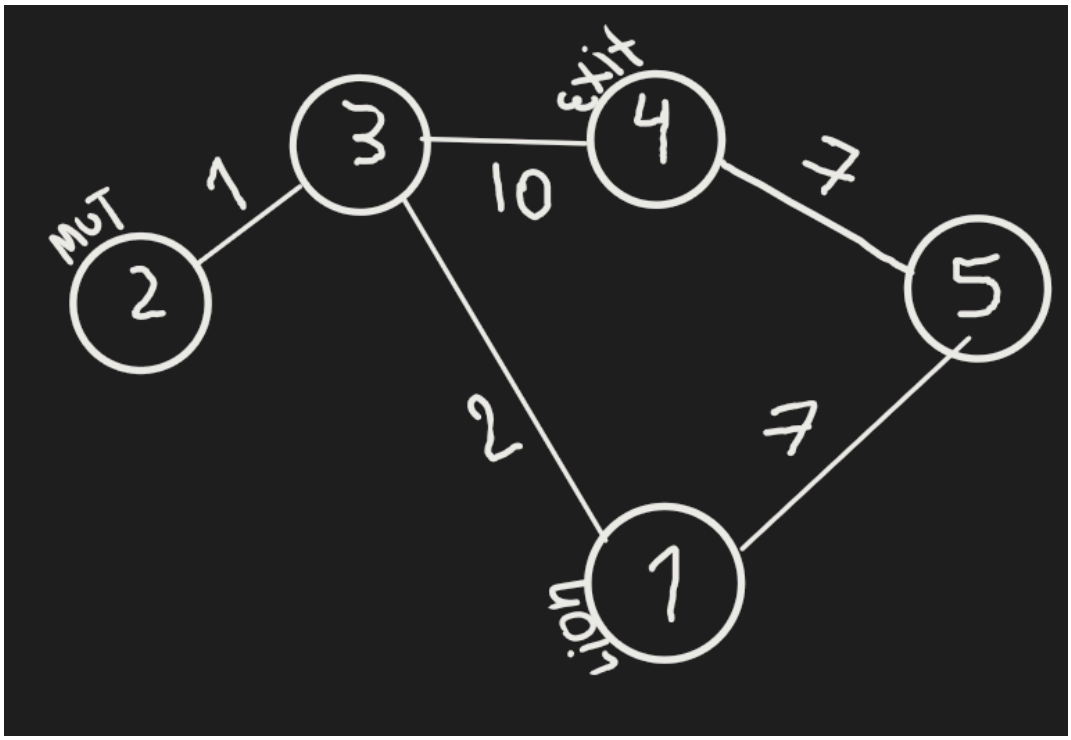
6 de noviembre:

En base a la retroalimentación brindada por el profesor y aunque los casos de prueba funcionen con la lógica inicial que desarrollamos, no es la más adecuada. Se nos plantea el hecho de que el camino más corto puede ser el que mayor velocidad requiera pese a la cercanía con los mutantes, la idea es priorizar encontrar el camino más óptimo sobre el más corto. En el contexto del problema, el camino más óptimo es aquel por el cual la velocidad garantiza ser la menor máxima posible. Es decir, si existe un camino A cuya velocidad mínima máxima para ser atravesado sin ser capturado es 150km/h, pero otro camino B garantiza 140km/h, entonces Lion debería priorizar imprimir la velocidad de B, incluso si B no es el camino más corto.

Es por ello que se realiza un nuevo caso de prueba bajo esta lógica. Se proponen 2 caminos para llegar a la salida, el primero es el más corto pero está conectado un nodo de inicio de los

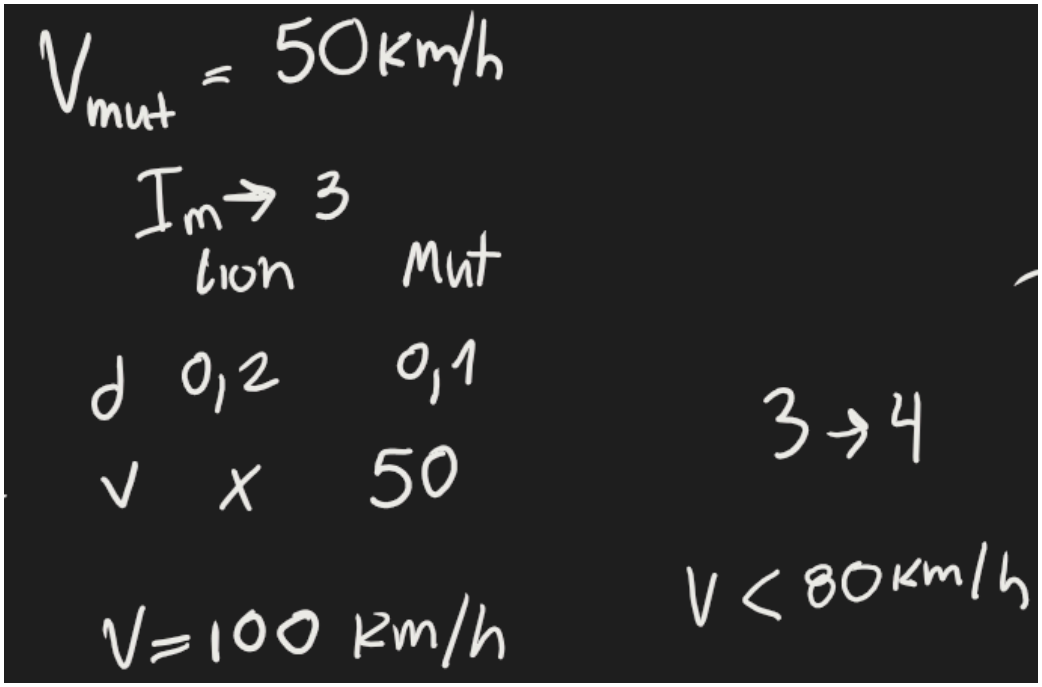
mutantes. El otro camino es un poco más largo pero no tiene conexión directa con el nodo de inicio de los mutantes.

Segundo caso de prueba. 5 Vertices, 5 Aristas, 1 salida espacial



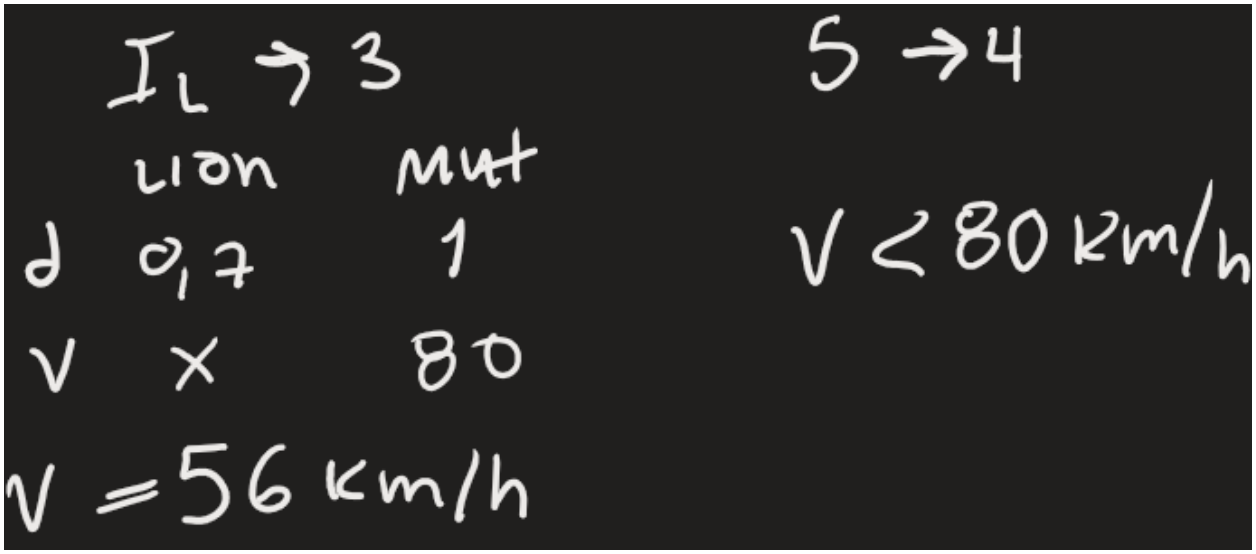
Siguiendo los parámetros conversados con el profesor, la distancia total del camino más corto del nodo de inicio de Lion no debe tomarse como único parámetro para la respuesta. Los fragmentos del camino más corto o subcaminos, son los que deben evaluarse para llegar a la salida espacial. En la nueva lógica, se propone que se debe aplicar la regla de 3 que propusimos en la lógica anterior, pero para cada sub camino del camino más corto. En este caso, 1-3-4 para lo cual

tenemos el siguiente análisis



Anteriormente se planteaba que despejar X indica la velocidad exacta, sin embargo, bajo la lógica indicada por el profesor de que si Lion y los mutantes llegan al mismo tiempo, Lion sería capturado, por tanto, la idea es que Lion tenga una velocidad mayor a la despejada, no mayor o igual. En base al planteamiento inicial que tuvimos y es descrito anteriormente, el programa debería arrojar ≈250.000000Km/h no estrictamente 250.

Este resultado es correcto bajo la lógica inicial, sin embargo, existe otra alternativa de Lion para llegar a la estación espacial, esta es por el camino 1-5-4. Este camino, a diferencia del anterior que costaba 1,2, cuesta 1,4. Cabe resaltar que el camino más óptimo en este nuevo razonamiento no es aquel que menos tiempo cuesta recorrer, si no el que menor velocidad máxima requiere. Por tanto el parámetro central para determinar cuál camino es mejor, es la velocidad. El camino por el cuál la velocidad sea menor, es el camino ideal para indicar el resultado según lo especificado en el problema. Bajo este razonamiento, realizamos el mismo proceso para el otro segmento:



Observamos que, este camino es más lejano de los mutantes, y esto permite que Lion avance en menor velocidad. Se asume que los mutantes llegan por el camino más corto 2-3-1-5. Comparando con la velocidad anterior, esta velocidad es 3 veces mejor que la anterior, aunque el tiempo aumente considerablemente.

Comparando el resultado de ambos tenemos V1max > V2max => 250Km/h >56km/h

Según este razonamiento, el resultado propuesto para el caso sería 56km/h. Sin embargo, es un estimado que deberá validarse con un algoritmo formal.

Ya que Lion no debería buscar el camino más corto, si no el más óptimo, se descarta que la respuesta sea un algoritmo base de SSSP. Se plantea buscar los caminos más cortos de Lion hacia todos los nodos que encaminan las estaciones espaciales y reconstruir un camino óptimo y seguro.

En los algoritmos naturales para SSSP estudiados en clase tenemos: BFS, DAGS, Bellman Ford-Moore y Dijkstra. Necesitamos encontrar todos los caminos que conducen desde la posición inicial de Lion hasta una estación espacial de manera óptima. Descartamos DAGS porque no necesariamente el grafo recibido es acíclico, también BFS porque las aristas no necesariamente contienen pesos del mismo valor, podríamos usar Bellman-Ford-Moore o Dijkstra, pero, ya que el problema no incluye ningún tipo de arista negativa y por cuestiones de complejidad, Dijkstra es una alternativa óptima, sin embargo, es necesario hacer modificaciones sobre su estructura nativa para que pueda ayudarnos en la solución.

Una modificación inicial planteada debería incluir un arreglo de tuplas o un arreglo asociado a las distancias donde indique el tiempo de llegada a cada nodo, no solo la distancia recorrida. Ya que no conocemos la velocidad de Lion, tampoco podemos saber el tiempo en el que recorre una distancia, por lo tanto, Lion haría Dijkstra para cada posibilidad de V que tenga y construirá la tupla.

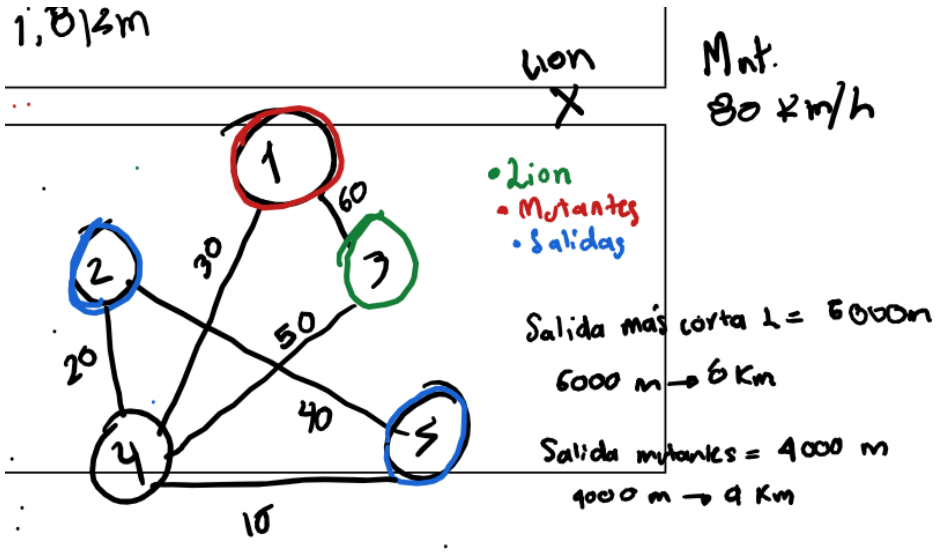
7 de noviembre

Se reevalúa el primer caso de prueba y se descarta la lógica inicial en base a lo charlado con el profesor, puesto a que los mutantes no mantienen un movimiento constante permanente (pudiendo desacelerar), se plantea el hecho de que los mutantes pueden esperar en un nodo específico, es necesario que Lion llegue a ese nodo antes que los mutantes.

Continuando la lógica de ayer y sumando este nuevo requisito, es evidente que se requiere un comparador o evaluador para cada recorrido. Puesto a que además de recorrer los caminos en un tiempo inferior al de los mutantes, hay que evaluar si un camino recorrido tuvo una velocidad máxima mayor que otro. Ya no solo bastaría simplemente con una regla de 3 para todos los sub caminos de un camino más corto, ahora necesitamos evaluar un verificador adicional que determine qué camino es más óptimo o necesita menor velocidad máxima.

Mediante la fórmula de distancia mencionada anteriormente y la tupla en Dijkstra, necesitamos determinar la velocidad mínima en la que se cumple Tiempo de llegada Mutantes > Tiempo de llegada Lion. Despejando sería: Distancia de Mutantes hacia un nodo V / Velocidad de mutantes > Distancia de Lion hacia un nodo V / Velocidad Prueba de Lion

Podríamos plantear nuevamente el caso de prueba visto anteriormente

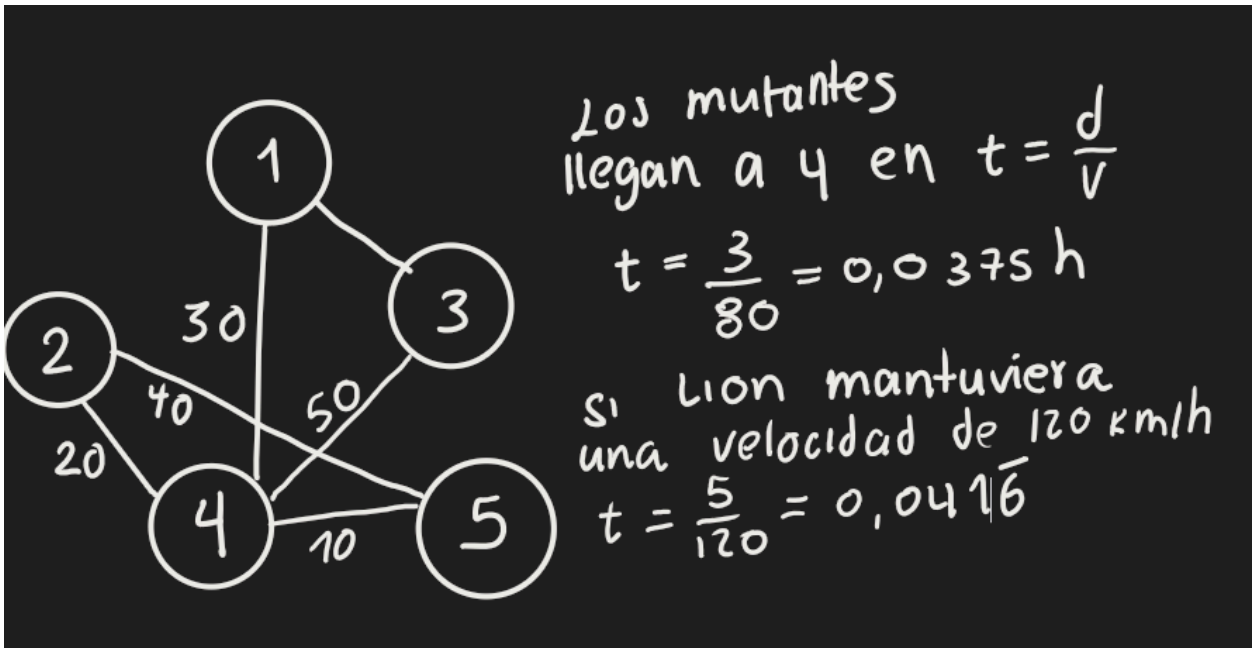


Bajo la lógica en ese momento, se podía obtener la velocidad de Lion simplemente despejando X en una regla de 3, donde teníamos las distancias de Lion y los mutantes, así como la velocidad de los mutantes. Operando tendríamos:

Velocidad de Lion = (Distancia Lion a E.E. * Velocidad mutantes) / Distancia Mutantes a E.E. con los datos del grafo sería V = (6 * 80) / 4 => V= 120.

Nótese que E.E. es la estación espacial más cercana encontrada (en la lógica de ese entonces) por el camino más corto que lleve hacia una estación espacial

Con la hipótesis inicial se obtuvo un valor de 120 km/h. Sin embargo. este cálculo no demostraba por sí solo un escape garantizado y se plantea el análisis de tramos por separado



Nótese que en el despeje de T se usa como distancia el valor directamente en Km después de haberse realizado la conversión correspondiente (d*100)/1000.
Ya que los mutantes podrían estratégicamente desacelerar y esperar a Lion, también pueden encaminarse en la ruta por la que Lion está llegando, haciendo que pierda de igual manera.

Y en este caso, el razonamiento desarrollado se obtuvo como valor candidato 133.33 km/h para llegar a una salida espacial, preferiblemente la del nodo 5. Nótese que esta velocidad se realiza en base a la lógica planteada y no tiene un fundamento asegurado como un valor correcto, se debe asegurar más adelante.

Donde d es la distancia de Lion y t el tiempo que tardan los mutantes en llegar. Obtenemos: v = 5 / 0,0375 = 133,333…km/h

9 de noviembre:

Se plantea con el profesor la idea de realizar para cada uno de los caminos de Lion hacia la salida, la búsqueda de la velocidad ideal, sin embargo, en la retroalimentación se indica que puede ser una alternativa costosa y debe ser replanteada. Posteriormente se sugiere realizar un Dijkstra con nodo de inicio de los mutantes hacia todos los nodos del grafo. Esta nueva idea concibe el hecho de que necesitamos saber la ubicación de los mutantes en determinados momentos para evaluar si Lion puede ingresar a ellos o es un lugar no seguro. Este nuevo planteamiento no busca evaluar qué velocidad es mejor, si no, si llegar a un nodo es seguro bajo una velocidad específica. La velocidad mínima que garantice recorrer un camino es la candidata a ser la respuesta.

Para ello planteamos tres etapas preliminares:

-1 Un Dijkstra con origen en la posición de inicio de los mutantes hacia todos los nodos del grafo

-2: El análisis de Lion sobre los caminos que son seguros de recorrer con una velocidad arbitraria en base en lo descrito en el paso 1.

-3: La revisión de la velocidad mínima necesaria para recorrer el camino completo hasta una estación espacial. Este comprobante no solo se basa en la mínima velocidad entre las demás usadas, si no si con esa velocidad es posible alcanzar la salida sin ser capturado o en otras palabras, es un recorrido seguro

10 de noviembre:

En los cambios realizados al código, es posible observar el verificador que se mencionó anteriormente para determinar un resultado inmediato para casos específicos. Por ejemplo, cuando Lion empieza en una salida espacial inmediatamente ganaría y no es necesario el algoritmo asociado necesario para salir, pues la velocidad para recorrer la distancia entre un nodo con sí mismo siempre será 0, pues la distancia es igualmente 0. Asimismo, se agrega un comprobante que, en dado caso que hay una única salida y esta coincide con la posición de inicio de los mutantes, será imposible que Lion gane, pues los mutantes lo podrían esperar en ella.

Adicionalmente, se plantea la impresión final para la cual, el algoritmo del problema arroja un valor ans que corresponde a la velocidad necesaria de Lion para salir. El algoritmo haría que este valor sea -1 para todo caso en el que Lion no pueda escapar. En esa instancia, el programa imprimirá que ha sido capturado. De lo contrario, incluyendo la instancia de iniciar una salida, siempre que ans sea mayor o igual a 0, significa que hay una velocidad posible y por tanto será el resultado del programa imprimiendo ans con 6 decimales.

Un cambio mínimo realizado es durante la creación del grafo, se tiene en primera instancia, las conversiones respectivas de los valores leídos en metros y luego a km. Asimismo, esta nueva implementación de las listas de adyacencia permite que el nodo procesado se guarde en su respectivo índice en la lista. Por ejemplo, las conexiones con el nodo 5 se almacenan en la posición 5 de la lista de nodos.

```
def minMaxSpeed():

      # Logica del algoritmo funcional conectando los algoritmos asociados

   return ans

def main():

   T = int(stdin.readline())

   while T:
```

```
        N,M,K,V = map(int,stdin.readline().split())
        intersc = [[] for i in range(N+1)]
        exits = []
        while M:
            A,B,D = map(int,stdin.readline().split())
             D = (D*100)/1000
            intersc[A].append((B,D))
            intersc[B].append((A,D))
            M -= 1
        S, R = map(int,stdin.readline().split())
        e = map(int,stdin.readline().split())
        for i in e:
            exits.append(i)
        T -= 1
        if S in exits:
            ans = 0
        elif (R in exits and len(exits)==1) or S == R:
            ans =-1
        else:
            ans = minMaxSpeed()
        if ans >= 0:
            print(f"Lion-O can escape at a speed of {ans:.6f} km/h!")
        else:
            print("Mumm-Ra captures Lion-O!")

main()
```

Por otro lado, se plantea una función calcTime, conectada con el Dijkstra sobre los mutantes, que permite observar el tiempo de llegada de los mutantes a todos los nodos, esto se logra gracias a la lista de caminos más cortos obtenidas mediante el Dijkstra. Se realiza un ciclo que va llenando el

arreglo time mediante cada distancia del arreglo, la velocidad de los mutantes y la fórmula de distancia despejando tiempo t = d/v

Más adelante, esto nos permitirá identificar si el momento en el que llega Lion a un nodo es seguro comparando los tiempos de llegada de los mutantes y él mismo.

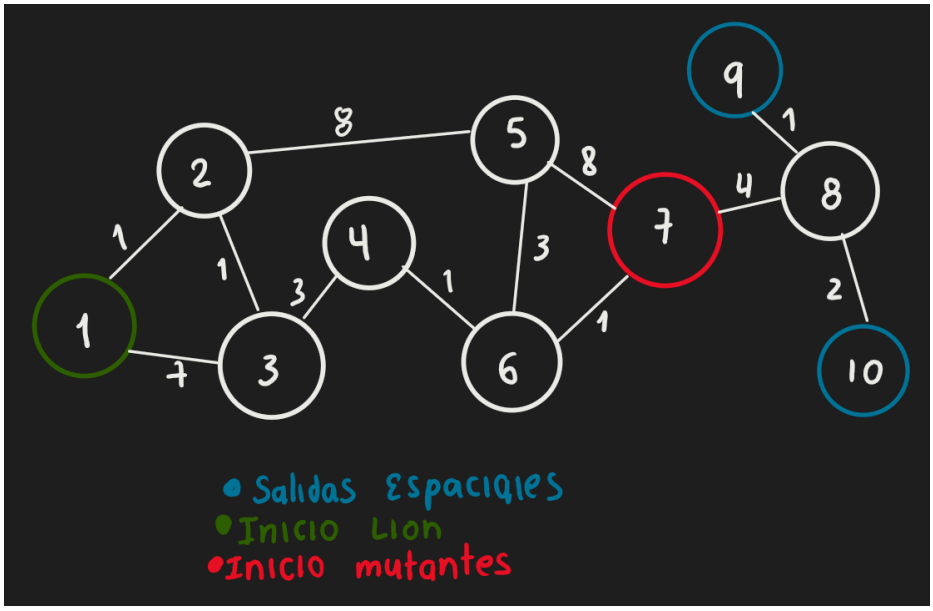
```
def calcTime(dist, V):

    time = [-1 for _ in range (len(dist))]

    for i in range(len(dist)):
        time[i] = (dist[i]/V)

    return time
```

Durante nuestra búsqueda de casos de prueba, específicamente un caso de captura, notamos que, para la gran mayoría de ellos, Lion tenía una velocidad mínima con la cual podía escapar. Esto solo se repetía en los casos donde había varios caminos directos hacia el nodo que contenía la salida espacial. Es decir, cuando había más de un nodo que tenía conexión a una salida espacial y Lion tenía acceso a ellos siempre y cuando no se produjera un efecto similar al cuello de botella en el que estaban los mutantes para acceder a él. Generalmente, los casos de captura inminente, son cuando hay una única forma de acceder a la salida espacial pero esta está bloqueada por los mutantes. Recordemos que los mutantes pueden acelerar y desacelerar, por tanto, si saben que Lion pasará por ese punto, no necesitan moverse ya que es obligatorio para acceder a la salida. Observemos el siguiente caso de prueba propuesto que arroja una captura:



En este caso observamos que, el único acceso a las salidas espaciales es mediante el nodo de inicio de los mutantes, este es el "cuello de botella" mencionado, los mutantes saben que Lion llegará a ese nodo, sin importar su velocidad, ni su tiempo, no necesitan moverse hacia 5 o 6, solo esperar. Es en el momento en el que Lion cruce por ese camino obligatorio a 7 que será capturado sin tener forma alguna de escapar.

11 de noviembre:

Se realizan las últimas correcciones previas a la primer entrega:

-Se adecúa la especificación del problema al primer esqueleto del algoritmo que se ha planteado y se mejora en base a las correcciones realizadas por el profesor.

-Se implementa el primer algoritmo de Dijkstra que permite obtener todas las distancias desde el nodo de inicio de los mutantes.

-Se conecta la funcionalidad de calcTime con el Dijkstra sobre el nodo de inicio de los mutantes

-Se organiza mejor el código y sus funciones. Asimismo se implementa el encabezado pertinente

-Refacciones en la redacción del documento